

GATEWEB · PROJECT TALK TRACK

## 專案表達稿

---

關網資訊 AI 應用工程師面談用：三個專案怎麼講、被追問怎麼接

版本 2：依你親口說明的真實分工修正（哪些是你做的、哪些是 AI 做的）

【請確認：…】的地方只有你知道實際細節，請用真實情況填上

# 怎麼用這份稿

## 三種長度，依場合挑

| 長度    | 什麼時候用                         |
|-------|-------------------------------|
| 一句話   | 自我介紹時帶過、HR 問「做過什麼」            |
| 30 秒版 | 面試官說「簡單介紹一下這個專案」              |
| 2 分鐘版 | 面試官說「詳細說說看」                   |
| 追問卡   | 被問「為什麼這樣設計」「遇過什麼問題」時，挑對應的卡片回答 |

## 三個原則

1. **不要背稿**：記住重點順序和關鍵數字，用自己的話講。
2. **講「為什麼」比講「用了什麼」重要**：就算設計不是你想的，你也要懂它為什麼這樣設計。
3. **誠實講分工**：AI 做的就說 AI 做的。被迫問時露餡，比一開始講清楚傷害大很多。

## 你的真實分工（面談一律以這張表為準）

| 專案                 | 你做的                                 | AI 做的                  | 面談時的定位                           |
|--------------------|-------------------------------------|------------------------|----------------------------------|
| 演唱會搶票系統            | 主結構；自己發現並修正兩個併發問題（資料庫沒有連線池、鎖票沒檢查結果） | 壓測腳本（全權由 AI 撰寫，你沒有修改）  | 最能證明你自己能力的專案，技術追問優先拿它回答          |
| Threat Intel Agent | 依照規劃 <b>實作</b> 、串接外部 API、部署上線       | <b>專案地圖與架構規劃</b>       | 展示「用 AI 協作開發 Agent」的經驗，以及你理解設計理由 |
| MineMarket         | 四人團隊隊長、認證模組、共用 decorator、部署（依履歷）    | 【請確認：有沒有使用 AI 協助、比例多少】 | 團隊合作與溝通的故事                       |

## 面談前一定要先處理的事

**重要** Threat Intel Agent 的 GitHub 版本和改版內容不一致。

- GitHub main 分支（8/11）與線上網站：舊版「兩輪架構」，也就是你履歷寫的版本。
- 本機 fix/hardening 分支（8/20）：新版「真正的迴圈、非同步、安全修正、70 個測試」，**沒有推上 GitHub**。

建議：面談以履歷上的兩輪架構為主。第二版的內容只有在你能自己解釋清楚時才提，而且要說明是透過 AI 協助審查改版的【請確認第二版是怎麼產生的】。

用語對照：避免誇大

| 不要說                           | 改成                                               |
|-------------------------------|--------------------------------------------------|
| 「我設計了 Threat Intel Agent 的架構」 | 「架構規劃是 AI 協助產生的，我負責實作、串接和部署，也去理解每個設計的理由」         |
| 「我寫壓力測試腳本」                    | 「壓測腳本是請 AI 寫的，用來驗證修正後的結果」                        |
| 「10 萬人搶票零超賣」                  | 「壓測模擬 10 ~ 600 人同時搶票，確認零超賣」                      |
| 「精通 RAG、MCP、LangGraph」        | 「理解原理與流程，還沒有上線專案，正在補實作」                          |
| 「串接過各家 LLM」                   | 「實際串接的是 Anthropic Claude；知道 OpenAI、Gemini 的格式差異」 |

## 1 分鐘自我介紹（把三個專案串起來）

面試官好，我是邱冠文，明新科技大學資訊工程系今年畢業。

我在當兵期間擔任新訓中心的教育班長四年多，負責規劃訓練課程、帶團隊。退伍後回到資工系，並花了一年半專心做後端專案。

我做過三個比較完整的專案：

第一個是**演唱會搶票系統**，主結構是我做的，用 FastAPI 和 Redis 處理大量使用者同時搶位；過程中我自己抓到兩個併發問題並修正，這是我學到最多的專案。

第二個是**四人團隊的電商後端**，我擔任隊長，負責架構切分和分工，也串了綠界的正式金流。

第三個是 **Threat Intel Agent**，我用 AI 協作的方式開發：架構規劃由 AI 協助產生，我負責實作 Claude 的 Tool Use、串接四個資安情資來源，並用 Docker 部署上線。

關網的電子發票系統需要高併發、零錯誤、重資安，現在又要把 AI Agent 落地到實際業務，這跟我累積的方向很接近，所以我很想加入。

**重點順序：**畢業背景 → 教育班長 → 三個專案一句話（誠實講分工）→ 連到關網。

**時間：**60~80 秒。

**為什麼要主動講 AI 協作：**AI 應用工程師的工作本來就會大量使用 AI 工具，先講清楚，後面被問「是不是 AI 寫的」就不會被動。

## 演唱會搶票系統（比賽作品暨畢業專題）

### 一句話

兩人協作的搶票系統，主結構與後端是我做的，用 FastAPI 和 Redis 鎖處理大量使用者同時搶同一個座位，過程中我自己找到並修正兩個併發問題。

### 30 秒版

搶票最大的問題是很多人同時搶同一個位子。我用 FastAPI 做後端、Redis 的 `SET NX` 鎖定座位，購票分成鎖位、Google Authenticator 雙重驗證、確認購買三段。

開發途中我發現兩個問題：資料庫沒有用連線池，跟 FastAPI 的非同步互相衝突；還有鎖票時沒有檢查鎖到底有沒有成功，人一多就開始搶成一團。兩個都修正後，再用壓測模擬 10 ~ 600 人同時搶票，確認零超賣。

## 2 分鐘版

**目標：**熱門場次開賣時，同一瞬間可能很多人搶同一個座位，系統不能把同一個位子賣給兩個人。

**我做的主結構：**FastAPI 加 Uvicorn 處理請求；座位用 Redis 的 `SET NX` 鎖定；購票流程分成鎖位、TOTP 驗證、確認購買；另外有會員系統與 Session 驗證。

### 我發現的問題一：資料庫沒有連線池

FastAPI 是非同步架構，一個 worker 本來可以同時處理很多請求，前提是等待資料庫時不能卡住。但我一開始沒有用連線池【請確認：原本是每個請求重新建立連線，還是大家共用同一條連線】，而且資料庫呼叫是同步的。人一多，請求就卡在建立連線或排隊等同一條連線上，非同步的好處被抵銷，兩者變成互相衝突。

修正：改用連線池重複使用連線【請確認：同時怎麼避免阻塞，是改用非同步驅動，還是把同步查詢交給 threadpool 執行】。

### 我發現的問題二：鎖票沒有檢查結果

`SET NX` 只有在這個座位還沒被鎖時才會設定成功，成功和失敗的回傳值不一樣。我一開始呼叫了它，**卻沒有把回傳值拿來判斷**，不管有沒有搶到鎖都繼續往下走，等於沒有鎖。人少時看不出來，人一多，兩個人同時搶同一個位子都能進入下一步，就開始競爭。

修正：檢查回傳值，**只有真的拿到鎖才繼續**，沒拿到就回應「座位已被選走」。

**驗證：**壓測腳本是請 AI 撰寫的，模擬 10 ~ 600 人同時搶票，確認修正後零超賣。【請確認：你有沒有自己執行壓測、看過結果】

**學到的：**非同步不是寫了 `async` 就會變快，整條路徑上只要有一個會卡住的地方就沒用；還有「呼叫了鎖」不等於「有鎖」，一定要檢查結果。

## 追問卡

### Q1：沒有連線池，為什麼會跟 FastAPI 非同步衝突？

FastAPI 用一個事件迴圈同時服務很多請求，靠的是「等待 I/O 時把執行權讓出來」。

如果每個請求都要重新建立資料庫連線，建立連線本身就很慢；如果大家共用同一條連線，就只能排隊一個一個查。再加上同步的資料庫呼叫會卡住事件迴圈，其他請求也跟著等。

連線池是預先建立一定數量的連線重複使用，不用每次重建，也不會全部擠在同一條連線上。

### Q2：鎖票的問題具體是什麼？怎麼修的？

我用 `SET NX` 鎖座位，它只在 `key` 不存在時才設定成功。問題是我沒有檢查它的回傳值，所以就算這個座位已經被別人鎖住、我的設定其實失敗了，程式還是繼續往下執行，好像自己搶到一樣。

修法是依回傳值判斷，成功才進入驗證和確認，失敗就直接告訴使用者座位已被選走。【請確認：鎖有沒有設定過期時間、多久】

### Q3：這兩個問題是怎麼發現的？

【請確認並用真實經過回答：是看到什麼現象（例如同一座位出現兩筆訂單、回應變得很慢、錯誤訊息）、在什麼情況下（人多、壓測時），怎麼一步步縮小範圍】

回答結構：現象 → 懷疑方向 → 怎麼確認 → 根因 → 修法 → 驗證。

### Q4：壓測是怎麼做的？

壓測腳本是我請 AI 撰寫的，我沒有修改內容。它模擬大量使用者同時送出鎖位與購票請求，最後檢查成功的訂單數有沒有超過座位數。

腳本內部的細節我沒有深入改過，如果要自己寫，我會【請確認你理解到哪裡：例如用 `asyncio` 或多程序同時送請求、統計成功與失敗數量、檢查資料庫訂單是否重複】。

### Q5：為什麼用 Redis 鎖，不用資料庫的鎖？

【請依你當時的理由調整】搶票是短時間內大量讀寫同一批座位，Redis 在記憶體運作、速度快，`SET NX` 本身是原子操作，適合做「搶到就先佔住」的短暫鎖定；確認購買後才寫進 MySQL 保存。

### Q6：跟關網有什麼關係？

電子發票系統每個月處理幾千萬張發票，同樣是高併發、不能出錯。像開立發票如果網路逾時後重試，就要確保不會重複開立，這跟搶票「一定要檢查結果、不能重複成功」是同一類問題。

## Threat Intel Agent ( 資安威脅情資 AI Agent )

### 一句話

輸入一個 IP 或網域，AI Agent 自己決定要查哪些資安情資來源，交叉比對後產出結構化的風險報告；這個專案我用 AI 協作開發，架構規劃由 AI 協助產生，我負責實作、串接與部署。

### 30 秒版

中小企業遇到可疑 IP，要手動查 WHOIS、VirusTotal、AbuseIPDB 再自己判斷，很花時間。

這個專案是用 AI 協作的方式做的：先請 AI 協助規劃專案架構，我依照規劃把程式實作出來，串接四個情資來源、用 Claude 的 Tool Use 讓模型決定要查哪些，最後強制模型用固定格式交報告，並用 Docker 部署到 Render。

實作過程中我也去弄懂每個設計為什麼這樣做。

### 2 分鐘版

**問題：**資安查證要比對好幾個來源、格式都不同，人工查一次要十幾分鐘。

**開發方式：**專案地圖和架構規劃是 AI 協助產生的；我負責依照規劃實作，包括串接 WHOIS、IPWHOIS、AbuseIPDB、VirusTotal、實作 Tool Use 流程、資料庫與 API、Docker 部署。

**這個架構的三個關鍵設計，以及我理解的理由：**

第一，**強制結構化輸出：**只靠 prompt 要求輸出 JSON 不夠穩定，所以第二輪用 `tool_choice` 強制模型呼叫 `submit_report`，欄位由 schema 定義，程式一定能解析。

第二，**快取：**VirusTotal 免費版每分鐘只有 4 次，所以用兩層裝飾器包住快取和計時，五個查詢工具共用，24 小時內重複查詢直接命中。

第三，**判斷和執行分開：**系統只判斷並寫入黑名單，提供 API 給防火牆查詢，不直接封鎖，避免 AI 誤判造成服務中斷。

**結果：**部署上線，有公開網址與 API 文件；每一步決策軌跡都存在 MongoDB，可以回溯判斷依據。

**我的收穫與限制：**我學會 Tool Use 的完整流程和怎麼讓 LLM 輸出穩定可用；但這個專案的架構不是我自己想出來的，設計取捨的能力是我現在正在補的。

### 追問卡

Q0：這個架構是你設計的嗎？

不是，專案地圖和架構規劃是我請 AI 協助產生的，我負責依規劃實作、串接、除錯和部署。

我覺得用 AI 協作開發是這個職位本來就會做的事，重點是要能判斷它給的設計合不合理、懂每個選擇的理由。例如為什麼要強制交報告、為什麼判斷和執行要分開，這些我都能說明。

#### Q1：為什麼沒有用 LangChain？

規劃時選擇直接用 Anthropic SDK，好處是每一步送了什麼給模型、模型回了什麼都很清楚，方便除錯。

我也理解框架的價值：流程有很多分支、需要記憶與追蹤時，LangChain 或 LangGraph 能省很多工，職缺提到的 LangGraph 是我要補的。

#### Q2：為什麼要強制呼叫 submit\_report，不直接叫它輸出 JSON？

模型輸出是機率性的，只靠 prompt 還是會偶爾輸出 Markdown 或缺欄位。用工具呼叫時，參數必須符合 schema，例如風險等級只能是固定幾個選項、證據一定是陣列，程式就不需要寫一堆容錯解析。

#### Q3：Tool Use 的流程你能講一次嗎？（實作是你做的，這題一定要會）

把使用者的目標和工具定義送給 Claude → Claude 回傳想呼叫的工具和參數 → 我的程式真正去呼叫外部 API → 把結果用 tool\_result 放回對話 → 最後強制 Claude 呼叫 submit\_report 交出結構化報告 → 存進資料庫、判定高風險就寫入黑名單。

#### Q4：快取怎麼運作？

兩層裝飾器：內層查快取、沒命中才真的打外部 API 並寫回；外層計時。Key 用「工具名稱 + 查詢目標」，避免不同工具互相覆蓋；TTL 24 小時。【請確認你能說明裝飾器怎麼把「結果、是否命中、耗時」三個值傳出來】

#### Q5：為什麼選 MongoDB？

各情資來源回傳的欄位差很多，報告裡的證據也是巢狀、不固定的結構，文件型資料庫比較自然。如果是發票、客戶這種結構固定、需要交易一致性的資料，會選 PostgreSQL。

#### Q6：為什麼判斷和執行要分開？

AI 一定會誤判，如果判定高風險就直接封鎖，一次誤判就可能擋掉正常客戶。所以系統只判斷並寫入黑名單，由防火牆透過 API 自己決定。放到關網也一樣：AI 可以協助判斷發票異常，但開立、作廢這種動作要有人確認。

#### Q7：如果繼續做，你會改什麼？

- 1. 建立標註好的測試集，量化判斷的準確率與一致性。
- 2. 現在的工具彼此獨立，模型其實不太需要多輪推理；要加入能互相牽引的工具，例如 IP 反查網域。
- 3. 流程變複雜時，改用 LangGraph 管理狀態與人工審核。

**第二版內容** 本機分支的改版（真正的迴圈、判斷門檻寫死、快取坑修正、前端金鑰與 XSS 修正、70 個測試）**只有在你能自己解釋時才提**，並說明是透過 AI 協助審查後改的【請確認】。講不清楚就不要主動提。

對應關網職缺

| 關網職缺寫的                        | 這個專案的對應（你實際做的部分）                    |
|-------------------------------|-------------------------------------|
| 協助開發維護 Tool Calling，串接第三方 API | 實作 Tool Use 流程，串接 4 個外部情資來源         |
| LLM API 對接、測試與異常處理            | 實作 Anthropic API 串接、外部 API 速率限制下的快取 |
| Python（FastAPI）微服務、Docker     | FastAPI 後端、Docker 部署到 Render        |
| 加分：資安或系統監控                    | 資安情資主題、決策軌跡紀錄                       |

## MineMarket 電商平台（四人團隊，擔任隊長）

**請確認** 以下依履歷整理。這個專案有沒有使用 AI 協助、哪些部分是你親手寫的，請先確認，再照 PART 0 的原則調整說法。

### 一句話

四人協作的遊戲道具電商後端，我當隊長，負責時程、會議、架構切分與分工，自己開發認證模組，也整合了綠界正式金流和遊戲伺服器即時發放道具。

### 30 秒版

這是四個人一起做的電商後端。我擔任隊長，用 Flask Blueprint 把系統切成認證、商品、購物車、後台四個模組，讓大家可以平行開發、減少合併衝突。

我負責認證模組，也寫了幾個全隊共用的 decorator。

金流串接綠界的正式環境，用 CheckMacValue 簽章驗證；付款成功後透過 Minecraft 的 RCON 即時發放道具，玩家不在線時由背景任務排隊補發。

### 2 分鐘版

**團隊面：**四個人程度不一，最大的風險是大家改到同一份檔案、互相覆蓋。所以我先規劃模組邊界，用 Blueprint 切成四塊，每個人負責一塊，共用的東西（路由守衛、參數解析、資料庫交易）寫成 decorator 給大家用。我也負責排時程、主持會議。

**技術面：**

- **認證模組：**會員註冊（含 Email 驗證、Cloudinary 上傳大頭貼）、登入、Session 驗證，並在資料庫層做單一裝置登入。
- **Decorator：**其中一個用 inspect 讀取函式參數，自動從請求組好參數傳進去。
- **金流：**串綠界正式金流，用 CheckMacValue 驗證通知確實來自綠界、內容沒被竄改。
- **即時發放：**訂單成立後透過 RCON 發道具；玩家離線時進入背景排隊，上線後補發。
- **部署：**Heroku 加 gunicorn。圖片放 Cloudinary，因為 Heroku 重啟會清空本機檔案；遊戲伺服器在本機，用 Ngrok 讓雲端連得到。

**學到的：**架構設計不只是技術決策，也要考慮團隊分工的邊界。

### 追問卡

Q1：CheckMacValue 是什麼？為什麼一定要驗證？

綠界付款完成會通知我們的伺服器，但任何人都能偽造「付款成功」的請求。所以要用雙方約定的金鑰，對參數依規則算出雜湊簽章，跟通知裡的 CheckMacValue 比對，一致才代表真的是綠界送的、內容沒被改。這跟電子發票 API、金流串接一樣：對外介面一定要驗證來源與完整性。

## Q2：玩家離線時的補發怎麼做？

【請依實作補充：佇列存在哪裡、多久檢查一次、怎麼避免重複發放】重點要講到同一筆訂單不管重試幾次，道具只能發一次。

## Q3：當隊長時遇過意見不合嗎？（也對應第三關情境題）

【請用真實事件，照 STAR 結構】情境 → 任務 → 你做了什麼（先聽理由、回到共同目標、用具體方式決定、決定後全力執行）→ 結果與學到的。

## Q4：為什麼用 Blueprint？

讓每個功能模組有自己的路由與邏輯，檔案分開，四個人能平行開發、減少 Git 衝突，也比較好找問題。

## 一定會遇到的通用追問

「這些程式碼是你自己寫的，還是 AI 寫的？」

參考回答（照你的真實分工）：

我會使用 AI 協作開發，但每個專案分工不一樣。

**搶票系統**的主結構是我自己做的，開發中我自己發現兩個問題：資料庫沒有用連線池、跟非同步衝突，還有鎖票時沒有檢查鎖定結果，人一多就互搶；這兩個都是我找出原因並修正的。壓測腳本則是請 AI 寫的。

**Threat Intel Agent** 是我請 AI 協助規劃架構，我依照規劃實作、串接和部署，並去理解每個設計的理由。

我覺得 AI 能讓開發變快，但工程師要能判斷它產出的東西對不對，這是我一直在練的能力。

可能的後續追問：「那你自己設計得出來嗎？」

小到中型的系統，像搶票系統的主結構我可以自己做；但像 Agent 這種我比較新的領域，目前還需要參考 AI 或文件來規劃，設計取舍的經驗是我正在累積的。搶票系統那兩個問題，就是我自己從現象找到根因的例子。

「你實際負責哪些部分？」

| 專案                 | 你的角色                                                |
|--------------------|-----------------------------------------------------|
| 搶票系統               | 兩人協作，主結構與後端；自己找出並修正兩個併發問題；壓測腳本由 AI 撰寫               |
| MineMarket         | 四人團隊隊長：時程、會議、架構切分、認證模組、共用 decorator、部署【請確認 AI 使用情況】 |
| Threat Intel Agent | 個人專案：架構由 AI 協助規劃，你負責實作、串接、部署                        |

「遇過最難的 bug 是什麼？」

首選：搶票系統的鎖票問題（完全是你自己的經驗）

**現象**：人少時正常，人一多就出現搶位混亂【請確認實際現象】。

**為什麼難找**：單人或少量測試時，鎖幾乎不會碰撞，完全看不出問題。

**根因**：呼叫了 SET NX，卻沒有檢查回傳值，沒搶到鎖的請求也繼續往下走。

**修法**：依回傳值判斷，拿到鎖才繼續。

**驗證**：壓測模擬 10 ~ 600 人同時搶票，確認零超賣。

**學到**：呼叫了鎖不等於有鎖，一定要檢查結果。

## 「你沒做過 RAG，進來怎麼辦？」

我還沒有 RAG 的上線專案，這是實話。但我理解完整流程：文件解析、清洗、切塊、embedding、存進向量資料庫；查詢時做相似度搜尋，必要時混合關鍵字檢索和 rerank，再把段落放進 prompt 要求依據回答；答錯時先分辨是檢索錯還是生成錯。

進來後我希望先從維護現有的 RAG pipeline、建立測試集開始。

## 「為什麼只串過 Claude？」

專案只需要一家，所以用了 Claude。我知道各家格式有差異，例如 Claude 的工具呼叫用 tool\_use 和 tool\_result 內容區塊，OpenAI 用 tool\_calls 和 role 為 tool 的訊息。要支援多家時，會定義共同介面、每家寫一個轉接器，並統一錯誤處理與備援切換。

## 「你的缺點是什麼？」

我實作和除錯的經驗，比架構設計的經驗多。像 Agent 這種比較新的領域，目前還會參考 AI 或文件來規劃。

我的做法是：不只把東西做出來，還要回頭弄懂每個設計為什麼這樣做、有哪些替代方案。搶票系統那兩個併發問題，就是我從「做出來」走到「真正理解」的過程。